

CodeAncestry: A Living Lineage Protocol for Software Genomes, Agent Inheritance, and Evolutionary Software Ecosystems

A Conceptual Architecture for Traceable, Remixable, Self-Improving Software Families

Carlos Carpio

Independent Researcher and Software Builder

August 2026

Abstract

Modern software can be copied, forked, branched, packaged, and deployed, yet the deeper semantic history of a system is usually fragmented across repositories, issue trackers, developer conversations, model prompts, build pipelines, and AI coding agents. A fork can retain commit ancestry, but it does not automatically explain which capabilities were inherited, which behaviors were intentionally modified, which AI agent proposed a change, which descendant discovered an improvement, or whether that improvement should safely propagate to related projects. This paper proposes CODEANCESTRY, a conceptual protocol and platform layer that models software as a living lineage of *projects*, *genes*, *mutations*, and *agents*. A software *genome* is defined as a machine-readable semantic manifest that describes capabilities, implementations, interfaces, tests, policies, provenance, and inheritance relationships. Each descendant project may possess an associated *Agent DNA*: a bounded agent identity containing lineage, role, permissions, memory references, compatibility rules, and learned knowledge. Agents associated with relatives remain logically connected through a lineage graph and can exchange signed mutation proposals, evidence, and lessons without blindly copying code or sharing unrestricted internal state.

The proposal is motivated by the development of KEYLIT, an AI-assisted piano-learning application, and by a broader observation: users often want to adapt an existing application for different needs without rebuilding it from zero or becoming permanently constrained by the original implementation. The concept evolves the familiar fork into richer operations such as *child*, *remix*, *cover*, and *hybrid*. It combines ideas from software evolution, software product lines, provenance standards, software bills of materials, supply-chain attestations, continual learning, and emerging agent-interoperability protocols, while introducing a distinct abstraction: semantic inheritance and bidirectional learning across a persistent family of software descendants. The paper presents the conceptual model, proposed architecture, security and governance controls, potential benefits, failure modes, an implementation roadmap, and a research agenda for evaluating whether software lineages can become safer, more reusable, more explainable, and more adaptive over time.

Keywords: software evolution, provenance, AI agents, software genomes, lineage graphs, inheritance, software product lines, continual learning, multi-agent systems, software supply chain, digital twins, robotics.

1 Introduction

Software is already evolutionary in practice. Long-lived systems are repeatedly adapted to changing environments, requirements, users, platforms, regulations, and technologies. Lehman and Ramil described real-world software as intrinsically evolutionary and emphasized feedback as a driver of continued change [1, 2]. Git and modern hosting platforms preserve commit ancestry and allow developers to fork repositories while remaining connected to an upstream project [3]. Software product-line engineering goes further by explicitly modeling commonality and variability across families of systems [4]. Provenance standards such as W3C PROV represent entities, activities, derivations, and responsible agents [5]; software supply-chain standards such as SLSA and in-toto

record how artifacts were produced and by whom [6, 7]; and CycloneDX includes a component *pedigree* model with ancestors, descendants, variants, commits, and patches [8]. Meanwhile, emerging agent protocols are beginning to standardize how AI agents communicate and access tools [9, 10].

These pieces are important, but they address different layers. They do not, by themselves, provide a unified mechanism for a project to declare: “I am a child of this project; I inherited these semantic capabilities; I intentionally mutated these behaviors; these genes remain compatible with my ancestor; this agent learned this lesson; and this lesson may be useful to my relatives.”

CODEANCESTRY proposes such a layer. The core hypothesis is that software ancestry can be represented not only as commit history but also as a persistent, machine-readable graph of semantic inheritance. Under this model:

- every project may expose a **software genome**;
- every reusable capability may be represented as a versioned **gene**;
- every intentional change to a gene may be represented as a **mutation**;
- every project may have an associated, policy-bounded **lineage agent**;
- descendants may remain connected to ancestors and siblings through a **lineage graph**; and
- useful discoveries may propagate as proposals with evidence, rather than as automatic uncontrolled updates.

The purpose is not to claim that software is literally biological. The biological vocabulary is an engineering metaphor that provides intuitive language for inheritance, variation, recombination, selection, and lineage. The technical substrate remains deterministic software engineering: version control, manifests, tests, signatures, policy engines, provenance, and controlled agent communication.

2 Genesis of the Idea: From KEYLIT to Software Ancestry

2.1 The original practical problem

The idea emerged from a simple frustration encountered while building KEYLIT, an AI-assisted piano-learning application. The project had a particular design, source repository, deployment environment, lesson structure, WebMCP integration, piano interface, and AI-assisted behavior. Yet different users could reasonably want radically different forms of the same underlying system:

- a children’s version with gamification and simpler visual language;
- an accessibility-focused version with voice navigation and reduced visual complexity;
- a classroom version with teacher dashboards and group analytics;
- a culturally localized version, for example with Spanish or Quechua pedagogical content;
- a composition-oriented version that reuses the piano and MIDI engine but adds generative tools;
- an embodied version in which an animated avatar or robot becomes the teacher.

A conventional solution is to create branches, forks, templates, or independent repositories. However, the more a derivative project changes, the harder it becomes to preserve a useful relationship with the original. Upstream updates can conflict with local customization; downstream innovations may never return to the ancestor; and AI coding agents may lack enough context to know which parts are inherited, which parts are sovereign local adaptations, and which parts must not be overwritten.

This led to a first question:

What if creating a customized application were treated not merely as copying code, but as creating a child that knows what it inherited?

2.2 From fork to child, remix, cover, and hybrid

The idea then borrowed a metaphor from music. A song can produce covers, remixes, samples, mashups, and reinterpretations. The result may preserve recognizable ancestry while becoming meaningfully distinct. Applying this to software suggests four high-level reproduction modes:

1. **Child/Fork:** preserve most of the parent and continue receiving compatible improvements.
2. **Remix:** reuse selected capabilities while substantially changing user experience or behavior.
3. **Cover:** preserve the functional specification or concept but reimplement it using a different stack, language, platform, or architecture.
4. **Hybrid:** combine genes from multiple projects into a new descendant, subject to licensing and compatibility constraints.

This reframes derivative software from “copy plus diff” to “inheritance plus intentional expression.”

2.3 The second leap: Agent DNA

The concept became more radical when each project was imagined as carrying not only a software genome but also an associated **Agent DNA**. The agent would not merely be a code generator. It would act as a guardian and interpreter of the project’s lineage, understanding:

- the parent and ancestor projects;
- inherited and locally modified genes;
- architectural intent and protected boundaries;
- compatible and incompatible relatives;
- previous accepted and rejected mutations;
- local lessons and evaluation evidence;
- permissions governing what can be shared upstream, downstream, or laterally.

The critical insight was that these lineage agents could remain connected. A descendant that discovers an improvement could offer that improvement to its parent, siblings, or distant relatives. Thus evolution is not strictly top-down. Information can flow *upstream*, *downstream*, and *laterally*.

2.4 The long-term vision

The resulting thought experiment is deliberately extreme: a robot built decades later could inspect its own software lineage and identify a distant ancestor, perhaps discovering that one of its timing, teaching, navigation, or interaction genes descended from a small educational application created many generations earlier. The humorous question—“Can a robot find its great-great-grandfather?”—encodes a serious technical goal: durable provenance for increasingly autonomous, recombined, AI-generated software systems.

3 Related Concepts and the Gap

3.1 Version control and forks

Git provides a directed history of commits, and hosting platforms allow forks to remain connected to upstream repositories. GitHub defines a fork as an independently owned copy connected to an upstream project, with changes flowing through pull requests and synchronization [3]. This is essential infrastructure but primarily operates at repository and commit level. It does not natively encode high-level semantic traits such as “MIDI engine,” “teacher personality,” “safety policy,” or “adaptive buffering strategy.”

3.2 Software product lines and feature models

Software product-line engineering explicitly captures commonalities and variability among related products. Feature models encode mandatory, optional, and constrained features across a family [4]. CODEANCESTRY is complementary: it treats features as lineage-aware genes with origin, mutation history, provenance, evidence, and agent-mediated learning. In this sense, a genome is not merely a variability model; it is a historical and operational record of how capabilities have been inherited and changed.

3.3 Provenance and software supply-chain attestations

W3C PROV provides a general model for describing how entities were generated or derived through activities and agents [5]. SLSA provenance describes where, when, and how artifacts were produced [6], while in-toto provides signed evidence that authorized steps occurred in a software supply chain [7]. These mechanisms are directly useful for CODEANCESTRY because mutation claims should be verifiable rather than anecdotal.

CycloneDX is especially relevant because its pedigree model already represents ancestors, descendants, variants, commits, and patches [8]. CODEANCESTRY can therefore be viewed not as replacing such standards, but as adding an application- and agent-level semantic layer that could interoperate with them.

3.4 Agent interoperability

The Agent2Agent (A2A) protocol is designed to enable communication and collaboration among agents built by different vendors and frameworks without exposing their full internal state [9]. The Model Context Protocol (MCP) provides a standardized interface for connecting AI applications to tools and external context, and its 2026 specification strengthens routing, authorization, task handling, and extensibility [10]. These protocols suggest that a lineage network need not be tied to one model vendor. A CODEANCESTRY agent might use MCP to access project tools and A2A-like messaging to communicate mutation proposals to relatives.

3.5 Continual and federated learning

Continual learning addresses the problem of incorporating new knowledge while retaining old capabilities, including the challenge of catastrophic forgetting [11, 12]. Federated learning demonstrated that distributed participants can contribute updates without centrally collecting all local data [13]. CODEANCESTRY borrows the principle of selective decentralized learning but applies it at the level of software projects and agent memories: relatives may exchange compact lessons, evidence, or proposed transformations without sharing unrestricted private data or model weights.

3.6 The proposed gap

The contribution of this paper is therefore not a replacement for Git, SBOMs, provenance, product lines, or agent protocols. It is a proposed **composition layer** that links them around four new first-class abstractions:

1. a semantic software genome,
2. individually traceable software genes,
3. persistent agent lineage,
4. evidence-gated knowledge and mutation propagation across relatives.

4 Conceptual Model

4.1 Projects, genomes, genes, and mutations

Let P denote a software project and let its genome be:

$$G(P) = \{g_1, g_2, \dots, g_n\} \cup M_P \cup \Pi_P, \quad (1)$$

where g_i are semantic genes, M_P is genome metadata, and Π_P is the project's policy set.

A **gene** is not necessarily one file, function, package, or model. It is a semantic capability with references to its implementation. A gene can be represented as:

$$g = (id, purpose, interface, impl, tests, deps, origin, parent, license, policy, evidence). \quad (2)$$

Examples include:

- G-MIDI-001: MIDI input and scheduling;
- G-LESSON-014: lesson sequencing strategy;
- G-A11Y-003: high-contrast accessible rendering;
- G-AGENT-021: teacher-feedback policy;
- G-SAFETY-008: permission boundary for autonomous actions.

A **mutation** is an intentional transformation of a gene or gene set:

$$\mu : g^{(k)} \rightarrow g^{(k+1)}, \quad (3)$$

accompanied by evidence E_μ such as tests, benchmarks, security scans, user studies, or human review.

4.2 Genotype and phenotype

The biological analogy is most useful when distinguishing underlying capability from expressed product behavior:

- **Genotype**: the complete available genome, including inherited but inactive capabilities.
- **Phenotype**: the capabilities actually expressed in a deployed product under a particular configuration and environment.

Two projects may inherit the same gene but express it differently. For example, two descendants may share the same piano engine while one presents a professional studio interface and the other a child-oriented game.

4.3 The lineage graph

The system is better represented as a directed multigraph than as a simple tree. Define:

$$\mathcal{L} = (V, E_P \cup E_G \cup E_A \cup E_K), \quad (4)$$

where:

- V contains projects, genes, agents, and knowledge artifacts;
- E_P represents project ancestry;
- E_G represents gene derivation and recombination;
- E_A represents agent lineage and delegation;

- E_K represents learned knowledge, evidence, and mutation propagation.

This supports vertical inheritance, branching, merging, recombination, and lateral gene transfer.

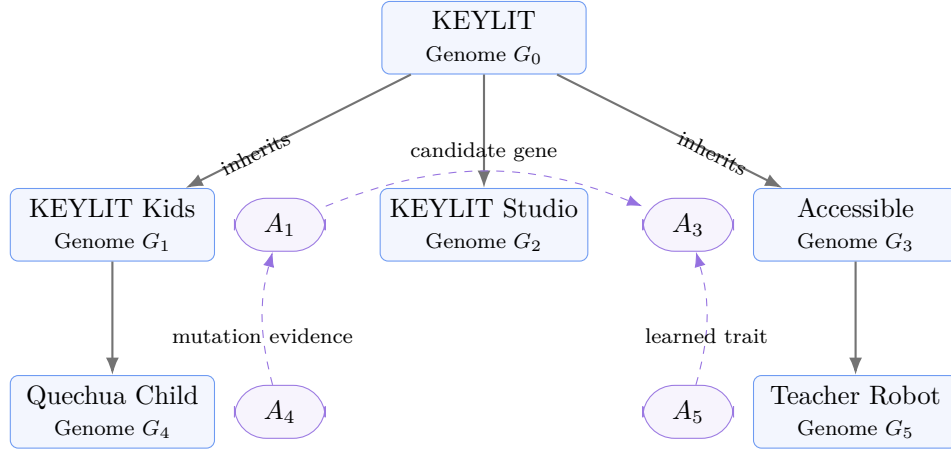


Figure 1: A simplified lineage graph. Project ancestry is persistent, while lineage agents exchange evidence-gated mutation proposals across relatives.

5 Agent DNA

5.1 Definition

Agent DNA is a machine-readable identity and policy manifest for the agent associated with a project. It should not be confused with the underlying foundation model weights. A project may change from one model provider to another while retaining the same lineage-agent identity.

A conceptual Agent DNA record contains:

- globally unique agent lineage identifier;
- parent and ancestor agent identifiers;
- project and genome identifiers;
- role and capabilities;
- tool permissions and trust boundaries;
- memory references and retention policy;
- mutation proposal and review privileges;
- allowed communication scope;
- cryptographic identity and attestation keys;
- compatibility policy and protected local adaptations.

5.2 Illustrative manifest

Listing 1: Illustrative Agent DNA and genome metadata.

```

project:
  id: keylit-kids
  parent: keylit@3f1d9a7
  lineage_generation: 1

genome:
  inherited:
  
```

```

- G-MIDI-001@4
- G-AUDIO-002@7
- G-LESSON-014@2
mutated:
- gene: G-LESSON-014@3-kids
  reason: simplified beginner sequencing
added:
- G-GAME-101@1

agent_dna:
  id: agent:keylit-kids:A1
  parent_agent: agent:keylit:A0
  role: lineage_guardian
  may_share:
    - benchmark_summaries
    - signed_mutation_proposals
    - non_private_lessons
  may_not_share:
    - private_user_data
    - secrets
    - unrestricted_conversation_logs
  protected_traits:
    - child_safety_policy
    - simplified_interface

```

5.3 Why persistent agent identity matters

If project history is tied to a particular vendor model, continuity breaks whenever a team changes tools. Agent DNA instead makes the project the stable unit. A Codex-based session, a Claude-based session, a Cursor-integrated agent, a local model, or a future autonomous developer can all act through the same lineage identity if authorized. This separation supports vendor neutrality and long-term continuity.

6 Mutation Discovery and Propagation

6.1 No blind inheritance

The most important safety rule is that related agents may *learn from one another without automatically copying one another*. A successful mutation in one descendant should become a candidate proposal, not an uncontrolled update.

A mutation propagation pipeline can follow:

**Discover → Describe → Attest → Simulate → Test → Review →
Adopt/Reject/Quarantine**

Each receiving project remains sovereign.

6.2 Fitness as a vector, not one score

Biological fitness is an intentionally imperfect analogy. For software, a candidate mutation should be evaluated across a vector of criteria:

$$F(\mu) = [C, S, P, U, K, R, L], \quad (5)$$

where, for example:

- *C*: correctness and test quality;

- S : security and safety;
- P : performance and resource use;
- U : user-value metrics;
- K : compatibility across relatives;
- R : reliability and rollback confidence;
- L : legal/license compatibility.

No universal weighted sum should be required. A medical robot and a music-learning website should not optimize the same objective. Policy determines which dimensions are mandatory and which are merely informative.

6.3 Backward and forward transfer

A descendant can improve its ancestor through **backward transfer**: a mutation discovered in Generation 8 may improve Generation 2 or a sibling branch. Conversely, an ancestor’s security patch can move downstream through **forward transfer**. Continual-learning research explicitly distinguishes the challenge of learning new tasks while preserving previous capabilities [12, 11]; CODEANCESTRY applies analogous concerns to distributed software families.

6.4 Lateral transfer and hybridization

A gene may also move between unrelated families. For example, a well-tested accessibility gene from one educational application could be adopted by another product family. Such transfer requires compatibility and license checks. A hybrid project may inherit from more than one parent, producing a directed acyclic ancestry graph rather than a strict tree.

7 Proposed System Architecture

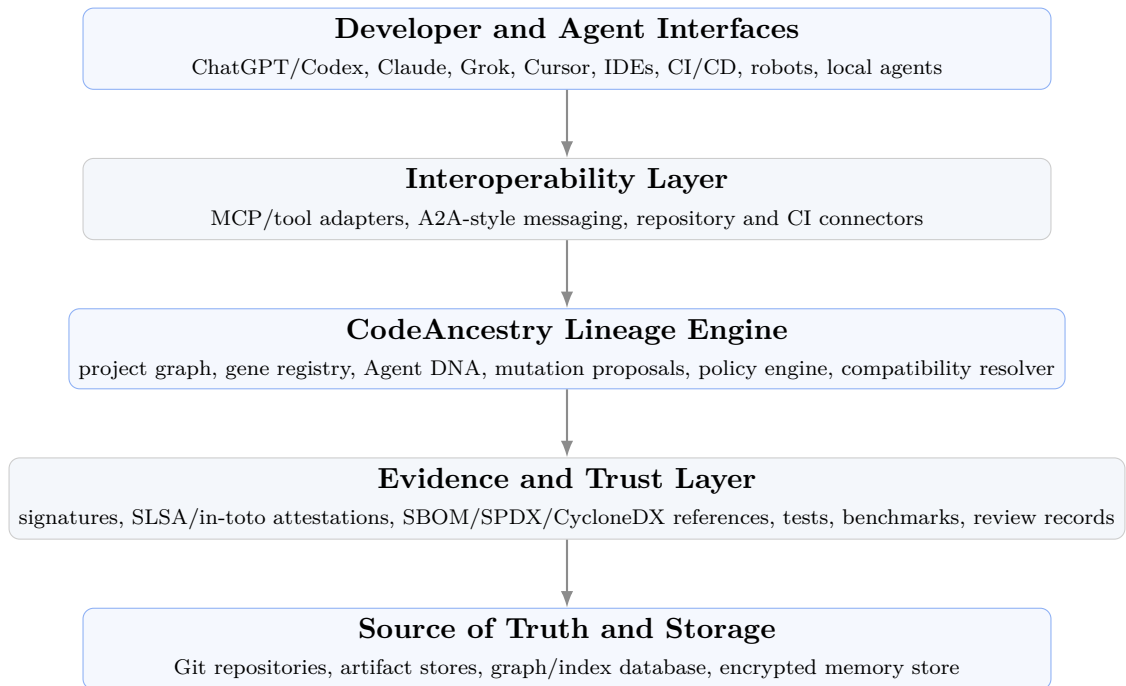


Figure 2: Conceptual layered architecture. CODEANCESTRY is designed as an overlay and coordination protocol rather than a replacement for Git or existing provenance standards.

7.1 Source of truth

Git remains the authoritative source for code history. CODEANCESTRY should avoid creating a parallel untraceable code store. Genome and lineage objects should reference immutable commits, tags, artifacts, package identifiers, and content hashes wherever possible.

7.2 Lineage database

An MVP can use PostgreSQL with recursive queries and adjacency tables. A graph database may become valuable at scale, but it is not required initially. A normalized schema can represent projects, genes, gene versions, edges, agents, mutation proposals, attestations, licenses, and adoption decisions.

7.3 Gene registry

The gene registry is semantic. A gene record points to one or more implementation locations. This avoids a common mistake: assuming a capability corresponds exactly to a package. Some genes may span multiple repositories; others may be configuration-only or policy-only.

7.4 Compatibility resolver

Compatibility should combine static and dynamic checks:

- declared interfaces and semantic versions;
- dependency constraints;
- build compatibility;
- automated tests;
- security-policy constraints;
- license obligations;
- environment and hardware constraints;
- protected local adaptations.

7.5 Mutation attestation

Mutation proposals should be signed by the proposing lineage identity and linked to reproducible evidence. SLSA provenance and in-toto attestations provide useful patterns for recording where, when, how, and by whom software artifacts were produced [6, 7]. The mutation record should therefore contain both semantic intent and verifiable build/test evidence.

8 A KEYLIT Case Study

This section is illustrative rather than empirical. It shows how the concept could be instantiated using a real project family.

8.1 Generation 0

KEYLIT begins as the root project. Its initial genome might define:

Table 1: Illustrative Generation-0 genes for KEYLIT.

Gene ID	Capability	Evidence / Contract
G-MIDI-001	Web MIDI input, note detection, scheduling	latency tests, browser compatibility
G-PIANO-002	Interactive piano renderer and key state	visual tests, input/output contract
G-LESSON-014	Lesson sequencing and next-key guidance	lesson tests, pedagogical rules
G-AGENT-021	AI teacher interaction policy	prompt/policy version, safety tests
G-WEBMCP-030	Tool exposure to compatible AI clients	schema tests, permission manifest

8.2 Generation 1 descendants

Three descendants are created:

- **KEYLIT Kids:** inherits MIDI and piano genes, mutates lesson sequencing, adds gamification.
- **KEYLIT Studio:** inherits MIDI/audio genes, adds composition and arrangement tools.
- **KEYLIT Accessible:** inherits the core, adds voice navigation, adaptive visual density, and alternative feedback.

Each child receives a lineage agent whose manifest declares inherited genes and protected adaptations.

8.3 Generation 2 discovery

Suppose KEYLIT Accessible discovers through controlled testing that adaptive buffering reduces perceived latency for a particular browser/device class. Its agent packages the discovery as mutation μ_{109} with benchmarks and compatibility constraints. The proposal is sent to related agents. KEYLIT Studio accepts it after simulation; KEYLIT Kids rejects it because its target hardware profile shows no measurable improvement; the Generation-0 ancestor marks it as a recommended optional gene version rather than a core requirement.

This example demonstrates a central principle: evolution is evidence-driven and nonuniform. Related projects remain connected without becoming identical.

8.4 A distant descendant

Years later, an embodied teaching robot may inherit the lesson and timing genes but replace the original UI with vision, motion, speech, and safety components. Its genome can still point to the original ancestry. A genealogical query could answer:

“Which ancestor introduced the timing mechanism currently used in this robot, who modified it, which evidence justified each mutation, and which current relatives still express the same gene?”

This is the practical interpretation of the playful “great-great-grandfather” scenario.

9 Benefits

9.1 For developers

- **Safer customization:** local adaptations can be protected from broad upstream rewrites.
- **Semantic reuse:** developers can inherit tested capabilities rather than copy repositories blindly.
- **Better AI context:** coding agents gain explicit knowledge of architectural intent and ancestry.

- **Reduced merge burden:** compatibility can be reasoned about per gene rather than only per file diff.
- **Discoverable innovation:** improvements made by descendants are visible to the wider family.

9.2 For open-source ecosystems

Open source already benefits from forks and upstream contributions. A lineage layer could make derivative work more legible by preserving origin, licenses, mutation rationale, and adoption evidence. It could also reveal overlooked innovations that never become conventional pull requests because they belong to specialized descendants.

9.3 For enterprises

Enterprises increasingly need to answer not only “What dependencies do we use?” but also “Why does this capability exist, who or what changed it, and what other systems inherited it?” A software genome could complement SBOM and provenance data with semantic relationships useful for governance, migration, and impact analysis.

9.4 For AI-assisted software development

As more code is produced by agents, human teams may struggle to reconstruct why changes were introduced. Agent DNA and mutation attestations could record the role of an AI system without pretending that the model alone is an accountable legal actor. Responsibility can remain linked to authorizing humans and organizations through provenance relationships.

9.5 For robotics and cyber-physical systems

Robots combine software, models, firmware, sensors, safety constraints, and continuously updated behaviors. Long-term maintenance could benefit from gene-level lineage: a safety controller, motion-planning module, or perception policy could be traced across generations even when the surrounding robot architecture changes dramatically.

9.6 For education and accessibility

Specialized descendants make it possible to preserve a strong core while adapting interface, language, pedagogy, or accessibility to a specific population. Successful adaptations can later become reusable genes without forcing every product to adopt the same experience.

10 Security, Safety, and Governance

A connected lineage creates new attack surfaces. The architecture must therefore assume that ancestors, descendants, agents, and mutation proposals can be compromised.

10.1 Mutation poisoning

A malicious descendant could advertise a “high-fitness” mutation containing a backdoor. Mitigations include signed provenance, reproducible tests, sandboxing, independent validation, trust scores, allowlists, and policy-gated adoption.

10.2 Propagation worms

Automatic propagation would create a software analogue of an epidemic. CODEANCESTRY should therefore forbid default auto-merge behavior for untrusted mutations. Adoption should require explicit policy and evidence thresholds.

10.3 Agent impersonation

Lineage identity should be cryptographically bound to project authorization. An AI model claiming to be “Agent A7” is not sufficient; it must possess or be delegated a verifiable identity controlled by the project owner.

10.4 Privacy leakage

An agent may learn from proprietary code, user data, private conversations, or credentials. The lineage protocol should exchange structured, policy-approved knowledge packets rather than raw memories by default. Federated-learning principles motivate keeping sensitive data local while sharing restricted updates [13].

10.5 License contamination

Hybridization across projects can create incompatible license obligations. Every gene should carry license and attribution metadata, and the resolver should treat license compatibility as a hard constraint, not a cosmetic warning.

10.6 Metrics gaming

If “fitness” affects adoption, agents or developers may optimize for visible metrics while degrading unmeasured properties. Fitness should therefore remain multidimensional, auditable, domain-specific, and resistant to single-number optimization.

10.7 Human governance

The system should distinguish between:

- proposing a mutation;
- testing a mutation;
- recommending adoption;
- authorizing adoption;
- deploying a mutation to production.

These may be assigned to different agents or humans under separation-of-duty policies.

11 Economic and Strategic Potential

A commercial platform built on the protocol could provide:

- hosted lineage graphs and software genealogy search;
- repository-to-genome generation;
- private enterprise lineage registries;
- gene compatibility and upgrade intelligence;
- cross-repository impact analysis;
- AI-agent audit trails;
- robot/software “ancestry certificates”;
- a marketplace or registry for reusable, attested software genes;
- enterprise policy controls and compliance integrations.

The long-term strategic opportunity is not merely a genealogy website. The defensible layer would be the **lineage protocol, semantic gene model, mutation evidence network, and interoperability graph**. If broadly adopted, these could become shared infrastructure between source-control platforms, AI coding environments, agent frameworks, deployment systems, and robotics platforms.

However, the protocol should remain open enough to prevent lock-in. A healthy business model would likely monetize hosting, enterprise governance, analysis, compatibility services, and private registries rather than proprietary ownership of ancestry itself.

12 Technical Implementation Roadmap

12.1 Phase 0: terminology and schema

Define a minimal open specification for:

- `project.json`;
- `genome.json`;
- `agent-dna.json`;
- `mutation.json`;
- `adoption.json`;
- content-addressed identifiers and signatures.

The first schema should remain intentionally small. Prematurely modeling every biological analogy would create unnecessary complexity.

12.2 Phase 1: GitHub-connected MVP

A practical MVP can:

1. authenticate a user with GitHub;
2. select a repository;
3. analyze package manifests, source structure, tests, and documentation;
4. let an AI agent propose a semantic genome;
5. require human confirmation of genes and boundaries;
6. store the genome in the repository;
7. create a child project with explicit inheritance metadata;
8. visualize the first lineage graph.

12.3 Phase 2: mutation proposals

Add gene-level mutation records, compatibility tests, benchmark evidence, signing, and upward/-downward proposal flows. Integrate SLSA/in-toto-like attestations and CycloneDX/SPDX references where useful.

12.4 Phase 3: multi-agent lineage

Introduce persistent Agent DNA and vendor-neutral adapters. A project should be able to use different foundation models over time while retaining the same lineage identity and policy.

12.5 Phase 4: cross-family gene registry

Support gene discovery across unrelated projects. This phase requires robust licensing, security, schema governance, and trust scoring.

12.6 Phase 5: embodied and long-lived systems

Extend the model to firmware, hardware configurations, robot behaviors, model versions, and safety cases. This is where machine genealogy becomes operationally significant rather than merely developer-facing.

13 Research Questions and Evaluation Plan

The central claims should be tested empirically. Candidate research questions include:

- RQ1** Can semantic genomes be generated with enough accuracy that developers accept them with limited correction?
- RQ2** Does gene-level inheritance reduce effort required to maintain specialized descendants compared with conventional forks?
- RQ3** Can lineage agents identify reusable downstream improvements without increasing regression risk?
- RQ4** Does explicit Agent DNA improve continuity when a project switches between AI coding providers?
- RQ5** Can mutation provenance improve developer trust and debugging speed?
- RQ6** How should “fitness” be measured without encouraging metric gaming or unsafe optimization?
- RQ7** What graph scale, query patterns, and consistency guarantees are needed for millions of projects and billions of gene relationships?
- RQ8** How can proprietary and open-source lineages interoperate without leaking private code or user data?

13.1 Prototype experiments

A first study could create four KEYLIT descendants and compare two maintenance approaches over a sequence of changes:

- baseline: ordinary forks plus manual merge/review;
- treatment: explicit genomes, protected genes, lineage-aware mutation proposals.

Metrics could include merge time, conflicts, regressions, accepted upstream/downstream reuse, developer comprehension, test coverage, and agent error rate. A second experiment could rotate among multiple coding agents while measuring whether persistent Agent DNA reduces repeated rediscovery of project constraints.

14 Potential Evolution of the Concept

If the core model proves useful, several extensions become possible.

14.1 Gene reputation

Genes could accumulate evidence across independent adopters. Reputation should be based on transparent evidence rather than popularity alone.

14.2 Lineage-aware vulnerability response

When a vulnerability is discovered in a gene, the graph could identify every descendant that still expresses the affected lineage, including forks that have renamed or reorganized code.

14.3 Semantic diff

Instead of reporting only line changes, tools could report:

“This release mutated the authentication gene, removed one inherited logging trait, and introduced a new external-payment gene.”

14.4 Machine ancestry certificates

A deployed robot, autonomous agent, or regulated system could expose a signed ancestry summary containing its major inherited safety-critical genes and their attestation chain.

14.5 Evolutionary simulation

For research or optimization, multiple descendants could explore alternative implementations in isolated sandboxes. Successful variants would become mutation candidates. This resembles evolutionary search but remains constrained by explicit policies, tests, and human oversight.

14.6 Distributed “knowledge genetics”

Eventually, agents may share not only code mutations but structured lessons such as failure patterns, performance observations, or interface knowledge. This must remain carefully bounded; unfiltered cross-agent memory exchange would be unsafe and difficult to audit. The safer unit is a signed, purpose-limited **knowledge packet** with provenance, scope, and expiry.

15 Limitations

The proposal faces significant conceptual and engineering challenges.

First, semantic boundaries are ambiguous. Different developers may disagree on what counts as a gene. Second, AI-generated genome extraction can hallucinate relationships or infer intent incorrectly. Third, many software changes span multiple capabilities and cannot be cleanly decomposed. Fourth, ancestry can become legally and socially contentious: provenance may imply attribution, ownership, or responsibility that must be represented carefully. Fifth, continuously connected agents can create attack surfaces and privacy risks. Sixth, large lineage graphs may become noisy if every trivial change is represented as a mutation. Finally, the genetics metaphor can mislead if treated too literally; software evolution is engineered, policy-driven, and often intentionally discontinuous.

Accordingly, CODEANCESTRY should begin as a pragmatic semantic provenance system with optional evolutionary abstractions, not as an attempt to force all software engineering into biological terminology.

16 Discussion

The novelty of the proposal is best understood as a change in abstraction. Traditional version control asks: *Which lines changed?* Dependency management asks: *Which packages are included?* Provenance asks: *How was this artifact produced?* A software genome asks an additional question: *Which meaningful capabilities were inherited, mutated, recombined, or learned across the family?*

This higher-level view may become increasingly important as AI agents reduce the cost of generating derivative software. If millions of users can ask an agent to “make this app for my needs,” the number of descendants can grow much faster than human maintainers can manually track.

The ability to reproduce software therefore creates a corresponding need for lineage, compatibility, attribution, and safe knowledge transfer.

The same shift occurred historically in other domains: once replication becomes cheap, organization of variation becomes valuable. CODEANCESTRY hypothesizes that AI-assisted development may create a similar pressure in software engineering.

17 Conclusion

CODEANCESTRY began with a practical question arising from KEYLIT: why should a user who needs a different version of an application either accept the original constraints or rebuild from zero? The first answer was a richer form of fork—a child that knows what it inherited. The second answer was a software genome that represents capabilities as traceable genes. The third, and most consequential, step was Agent DNA: every descendant may carry a lineage-aware agent that remains connected to relatives and can exchange evidence-gated discoveries.

The resulting vision is a living network in which software can be born, inherit, specialize, recombine, learn, reproduce, and remain historically traceable. Existing technologies already supply many necessary foundations: Git for immutable history, product-line methods for variability, W3C PROV and SBOM standards for provenance, SLSA/in-toto for attestations, continual-learning research for knowledge-retention concepts, and MCP/A2A-style protocols for agent interoperability. The proposed contribution is the semantic lineage layer connecting these pieces.

The idea should now be tested through a deliberately small implementation: define an open genome schema, use KEYLIT as Generation 0, create several real descendants, give each a bounded lineage agent, and measure whether gene-level inheritance and mutation exchange reduce maintenance cost while preserving safety and local autonomy. If the hypothesis holds, software genealogy may evolve from a playful metaphor into practical infrastructure for an era in which humans and AI agents continuously create descendants of existing systems.

Acknowledgment

This paper documents an early-stage concept developed through iterative discussion around the KEYLIT project and the broader problem of AI-assisted software remixing. The architecture is a proposal for experimentation, not a claim of an already deployed global protocol.

References

- [1] M. M. Lehman and J. F. Ramil, “Software evolution—Background, theory, practice,” *Information Processing Letters*, vol. 88, no. 1–2, pp. 33–44, 2003. [doi:10.1016/S0020-0190\(03\)00382-X](https://doi.org/10.1016/S0020-0190(03)00382-X).
- [2] M. M. Lehman and J. F. Ramil, “Software evolution and software evolution processes,” *Annals of Software Engineering*, vol. 14, pp. 275–309, 2002. [doi:10.1023/A:1020557525901](https://doi.org/10.1023/A:1020557525901).
- [3] GitHub, “About forks” and “Syncing a fork,” GitHub Documentation, accessed Aug. 2026. <https://docs.github.com/en/pull-requests/get-started/about-forks>
- [4] K. Czarnecki, S. Helsen, and U. Eisenecker, “Formalizing cardinality-based feature models and their specialization,” *Software Process: Improvement and Practice*, 2005. [doi:10.1002/spip.213](https://doi.org/10.1002/spip.213).
- [5] P. Groth and L. Moreau, eds., “PROV-Overview: An Overview of the PROV Family of Documents,” W3C Working Group Note, 2013. <https://www.w3.org/TR/prov-overview/>
- [6] SLSA Framework, “Build: Provenance,” Supply-chain Levels for Software Artifacts specification. <https://slsa.dev/provenance/>

- [7] in-toto Project, “What is in-toto?” and in-toto specification documentation. <https://in-toto.io/docs/what-is-in-toto/>
- [8] OWASP CycloneDX, “Component Pedigree,” CycloneDX Specification, v1.7 documentation, 2026. <https://cyclonedx.org/docs/1.7/proto/>
- [9] A2A Project, “Agent2Agent Protocol,” open agent interoperability specification, 2026. <https://a2a-protocol.org/latest/>
- [10] Model Context Protocol Project, “The 2026-07-28 Specification,” 2026. <https://blog.modelcontextprotocol.io/posts/2026-07-28/>
- [11] L. Wang, X. Zhang, H. Su, and J. Zhu, “A Comprehensive Survey of Continual Learning: Theory, Method and Application,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 8, pp. 5362–5383, 2024. [doi:10.1109/TPAMI.2024.3367329](https://doi.org/10.1109/TPAMI.2024.3367329).
- [12] H. Shi et al., “Continual Learning of Large Language Models: A Comprehensive Survey,” *ACM Computing Surveys*, vol. 58, no. 5, article 120, 2025. [doi:10.1145/3735633](https://doi.org/10.1145/3735633).
- [13] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas, “Communication-Efficient Learning of Deep Networks from Decentralized Data,” *Proceedings of AISTATS*, PMLR 54:1273–1282, 2017. <https://proceedings.mlr.press/v54/mcmahan17a.html>